

LlamaGuard NAV – CRE Workflow Onchain Contracts

Chainlink Labs & LlamaRisk

HALBORN

LlamaGuard NAV – CRE Workflow Onchain Contracts - Chainlink Labs & LlamaRisk

Prepared by:  HALBORN

Last Updated 02/05/2026

Date of Engagement: January 16th, 2026 - January 22nd, 2026

Summary

100% ⓘ OF ALL REPORTED FINDINGS HAVE BEEN ADDRESSED



TABLE OF CONTENTS

1. Introduction
2. Assessment summary
3. Test approach and methodology
4. Risk methodology
5. Scope
6. Assessment summary & findings overview
7. Findings & Tech Details
7.1 Price answer stored without validation
7.2 Market authorization not enforced on parameter retrieval
7.3 Additional data stored without format validation
7.4 Stale report processing without expiration validation
7.5 Oracle write access not validated during initialization
7.6 Missing cross-parameter validation on individual setters
7.7 Update type removal functionality missing
7.8 Report validation cannot be disabled

1. Introduction

LlamaRisk engaged Halborn to conduct a comprehensive security audit of the LlamaGuard Oracle smart contract ecosystem and the integration with the Horizon Agent and Chainlink CRE (Chainlink Runtime Environment) ecosystem. The primary aim was to assess the security architecture of the oracle contracts and parameter registry to identify vulnerabilities, evaluate existing security protocols, and provide actionable recommendations to enhance the security and operational efficacy of the Chainlink oracle framework. The assessment focused exclusively on the smart contracts provided, ensuring a thorough evaluation of their security mechanisms and integration patterns.

2. Assessment Summary

Our engagement with LlamaRisk spanned a 3-day period, during which we dedicated one full-time security engineer equipped with extensive experience in blockchain security, oracle architecture, and Chainlink protocol implementations. The objectives of this assessment were to:

- Verify the correct functionality of oracle round data management and risk parameter tracking.
- Identify potential security vulnerabilities within the LlamaGuardOracle, LlamaGuardOracleProxy, ParameterRegistry, AbstractCreReceiver contracts and any integrating contract such as HorizonFreezeAgent, BaseHorizonAgent and HorizonAgentHub.
- Evaluate the integration of Chainlink CRE workflows and access control mechanisms.
- Provide recommendations to enhance the security, reliability, and trustworthiness of the oracle ecosystem.

3. Test Approach And Methodology

Our testing strategy employed a blend of manual and automated techniques to ensure a thorough evaluation of the contract ecosystem. The assessment combined logical analysis, implementation review, and functional testing to uncover vulnerabilities across multiple risk vectors including access control, data validation, oracle feed integrity, and integration patterns. The testing process included:

- Detailed examination of the LlamaGuardOracle aggregator interface implementation and Chainlink AggregatorV3 compatibility.
- Comprehensive manual code reviews of risk parameter update mechanisms and access control enforcement.
- Analysis of metadata validation and CRE report processing within AbstractCreReceiver.
- Functional testing of price update flows, market authorization, and parameter registry operations.
- Integration verification of oracle proxy interactions with underlying oracle contracts and Horizon Agent.
- Assessment of data validation gaps and their potential impact on oracle feed integrity.

This executive summary encapsulates the pivotal findings and recommendations from our security assessment of the Chainlink LlamaGuard Oracle ecosystem. By addressing the identified issues and implementing the recommended fixes, LlamaRisk can significantly enhance the security, reliability, and trustworthiness of its oracle platform.

4. RISK METHODOLOGY

Every vulnerability and issue observed by Halborn is ranked based on **two sets of Metrics** and a **Severity Coefficient**. This system is inspired by the industry standard Common Vulnerability Scoring System.

The two **Metric sets** are: **Exploitability** and **Impact**. **Exploitability** captures the ease and technical means by which vulnerabilities can be exploited and **Impact** describes the consequences of a successful exploit.

The **Severity Coefficients** is designed to further refine the accuracy of the ranking with two factors: **Reversibility** and **Scope**. These capture the impact of the vulnerability on the environment as well as the number of users and smart contracts affected.

The final score is a value between 0-10 rounded up to 1 decimal place and 10 corresponding to the highest security risk. This provides an objective and accurate rating of the severity of security vulnerabilities in smart contracts.

The system is designed to assist in identifying and prioritizing vulnerabilities based on their level of risk to address the most critical issues in a timely manner.

4.1 EXPLOITABILITY

ATTACK ORIGIN (AO):

Captures whether the attack requires compromising a specific account.

ATTACK COST (AC):

Captures the cost of exploiting the vulnerability incurred by the attacker relative to sending a single transaction on the relevant blockchain. Includes but is not limited to financial and computational cost.

ATTACK COMPLEXITY (AX):

Describes the conditions beyond the attacker's control that must exist in order to exploit the vulnerability. Includes but is not limited to macro situation, available third-party liquidity and regulatory challenges.

METRICS:

EXPLOITABILITY METRIC (M_E)	METRIC VALUE	NUMERICAL VALUE
Attack Origin (AO)	Arbitrary (AO:A) Specific (AO:S)	1 0.2
Attack Cost (AC)	Low (AC:L) Medium (AC:M) High (AC:H)	1 0.67 0.33
Attack Complexity (AX)	Low (AX:L) Medium (AX:M) High (AX:H)	1 0.67 0.33

Exploitability E is calculated using the following formula:

$$E = \prod m_e$$

4.2 IMPACT

CONFIDENTIALITY (C):

Measures the impact to the confidentiality of the information resources managed by the contract due to a successfully exploited vulnerability. Confidentiality refers to limiting access to authorized users only.

INTEGRITY (I):

Measures the impact to integrity of a successfully exploited vulnerability. Integrity refers to the trustworthiness and veracity of data stored and/or processed on-chain. Integrity impact directly affecting Deposit or Yield records is excluded.

AVAILABILITY (A):

Measures the impact to the availability of the impacted component resulting from a successfully exploited vulnerability. This metric refers to smart contract features and functionality, not state. Availability impact directly affecting Deposit or Yield is excluded.

DEPOSIT (D):

Measures the impact to the deposits made to the contract by either users or owners.

YIELD (Y):

Measures the impact to the yield generated by the contract for either users or owners.

METRICS:

IMPACT METRIC (M_I)	METRIC VALUE	NUMERICAL VALUE
Confidentiality (C)	None (C:N)	0
	Low (C:L)	0.25
	Medium (C:M)	0.5
	High (C:H)	0.75
	Critical (C:C)	1
Integrity (I)	None (I:N)	0
	Low (I:L)	0.25
	Medium (I:M)	0.5
	High (I:H)	0.75
	Critical (I:C)	1
Availability (A)	None (A:N)	0
	Low (A:L)	0.25
	Medium (A:M)	0.5
	High (A:H)	0.75
	Critical (A:C)	1
Deposit (D)	None (D:N)	0
	Low (D:L)	0.25
	Medium (D:M)	0.5
	High (D:H)	0.75
	Critical (D:C)	1
Yield (Y)	None (Y:N)	0
	Low (Y:L)	0.25
	Medium (Y:M)	0.5
	High (Y:H)	0.75
	Critical (Y:C)	1

Impact I is calculated using the following formula:

$$I = \max(m_I) + \frac{\sum m_I - \max(m_I)}{4}$$

4.3 SEVERITY COEFFICIENT

REVERSIBILITY (R):

Describes the share of the exploited vulnerability effects that can be reversed. For upgradeable contracts, assume the contract private key is available.

SCOPE (S):

Captures whether a vulnerability in one vulnerable contract impacts resources in other contracts.

METRICS:

SEVERITY COEFFICIENT (<i>C</i>)	COEFFICIENT VALUE	NUMERICAL VALUE
Reversibility (<i>r</i>)	None (R:N) Partial (R:P) Full (R:F)	1 0.5 0.25
Scope (<i>s</i>)	Changed (S:C) Unchanged (S:U)	1.25 1

Severity Coefficient *C* is obtained by the following product:

$$C = rs$$

The Vulnerability Severity Score *S* is obtained by:

$$S = min(10, EIC * 10)$$

The score is rounded up to 1 decimal places.

SEVERITY	SCORE VALUE RANGE
Critical	9 - 10
High	7 - 8.9
Medium	4.5 - 6.9
Low	2 - 4.4

SEVERITY	SCORE VALUE RANGE
Informational	0 - 1.9

5. SCOPE

REPOSITORY
(a) Repository: llamaguard-contracts (b) Assessed Commit ID: 40b9ec8 (c) Items in scope: - src/ParameterRegistry.sol - src/LlamaGuardOracle.sol - src/abstracts/AbstractCreReceiver.sol - src/horizon-response/agents/HorizonFreezeAgent.sol - src/horizon-response/agents/BaseHorizonAgent.sol - src/LlamaGuardOracleProxy.sol - src/interfaces/ILlamaGuardOracle.sol - src/abstracts/AbstractReadWriteAccessController.sol - src/horizon-response/HorizonAgentHub.sol
REMEDATION COMMIT ID:
https://github.com/llama-risk/llamaguard-contracts/pull/19 https://github.com/llama-risk/llamaguard-contracts/pull/19/ https://github.com/llama-risk/llamaguard-contracts/pull/19/changes/d95ab291e164990a7fd9eb25aa8bd7744fa9bccc
Out-of-Scope: New features/implementations after the remediation commit IDs.

6. ASSESSMENT SUMMARY & FINDINGS OVERVIEW



SECURITY ANALYSIS	RISK LEVEL	REMEDIATION DATE
PRICE ANSWER STORED WITHOUT VALIDATION	LOW	SOLVED - 01/29/2026
MARKET AUTHORIZATION NOT ENFORCED ON PARAMETER RETRIEVAL	LOW	SOLVED - 01/29/2026
ADDITIONAL DATA STORED WITHOUT FORMAT VALIDATION	LOW	SOLVED - 01/29/2026
STALE REPORT PROCESSING WITHOUT EXPIRATION VALIDATION	LOW	SOLVED
ORACLE WRITE ACCESS NOT VALIDATED DURING INITIALIZATION	INFORMATIONAL	ACKNOWLEDGED - 01/29/2026
MISSING CROSS-PARAMETER VALIDATION ON INDIVIDUAL SETTERS	INFORMATIONAL	ACKNOWLEDGED - 01/29/2026
UPDATE TYPE REMOVAL FUNCTIONALITY MISSING	INFORMATIONAL	SOLVED - 01/29/2026
REPORT VALIDATION CANNOT BE DISABLED	INFORMATIONAL	ACKNOWLEDGED - 01/29/2026

7. FINDINGS & TECH DETAILS

7.1 PRICE ANSWER STORED WITHOUT VALIDATION

// LOW

Description

The `updateLatestRiskRoundData()` function in `LlamaGuardOracle` decodes the `newValue` parameter as an `int256` price and immediately stores it in round data via `_updateLatestRoundData()` without any validation. The decoded price is not checked for logical bounds such as negative values for assets that should only have positive prices, extreme values approaching `type(int256).min` or `type(int256).max`, or unreasonable deviations from the previously stored price. This allows arbitrary price values to be permanently committed to the oracle's round history and emitted in events, potentially poisoning downstream systems that rely on this oracle for accurate pricing data. An authorized `WRITER_ROLE` account could intentionally or accidentally set nonsensical prices that corrupt the oracle feed.

BVSS

AO:A/AC:L/AX:L/C:N/I:L/A:N/D:N/Y:N/R:N/S:U (2.5)

Recommendation

Implement strict validation on the decoded price before storing it. Add checks to ensure the price is within acceptable bounds, such as requiring it to be positive for assets that should never have negative prices, and optionally validating price movements against the previous round to detect suspicious jumps. Consider also adding a configurable `maxPriceDeviation` parameter that allows administrators to set thresholds for acceptable price changes between consecutive updates, reverting if a new price deviates excessively from the previous value.

Remediation Comment

SOLVED: The LlamaRisk team solved this finding.

Remediation Hash

<https://github.com/llama-risk/llamaguard-contracts/pull/19>

7.2 MARKET AUTHORIZATION NOT ENFORCED ON PARAMETER RETRIEVAL

// LOW

Description

The `getLatestUpdateByParameterAndMarket()` function in `LlamaGuardOracle` accepts a market address parameter and returns the latest risk parameter update for that market, but does not validate whether the provided market is actually authorized. While the contract implements `addAuthorizedMarket()` and `removeAuthorizedMarket()` functions with `DEFAULT_ADMIN_ROLE` access, suggesting that market authorization is a security control, this validation is never applied in the retrieval function or any other. This allows any caller to query parameter updates for unauthorized or non-existent markets, potentially enabling information disclosure or allowing downstream systems to act on data they should not have access to if market-based access control is part of the intended security model.

BVSS

AO:A/AC:L/AX:L/C:N/I:L/A:N/D:N/Y:N/R:N/S:U (2.5)

Recommendation

Add a market authorization check in `getLatestUpdateByParameterAndMarket()` before returning the update. The function should require that the provided market address is authorized by calling `require(_authorizedMarkets[market], UnauthorizedMarket(market))`. Additionally, ensure that the same validation is applied in any other functions that retrieve or filter data by market to maintain consistent enforcement of the authorization model across the contract.

Remediation Comment

SOLVED: The LlamaRisk team solved this finding.

Remediation Hash

<https://github.com/llama-risk/llamaguard-contracts/pull/19>

7.3 ADDITIONAL DATA STORED WITHOUT FORMAT VALIDATION

// LOW

Description

The `LlamaGuardOracle.updateLatestRiskRoundData()` function stores the `additionalData` field from the `UpdateInput` directly into `updateHistory` without validating its encoding format or content. Downstream consumers like `HorizonFreezeAgent` expect `additionalData` to be ABI-encoded with specific types and values (e.g., two `int256` bounds and a `uint256` freeze state), but there is no enforcement at storage time. This allows malformed or incorrectly encoded `additionalData` to be permanently committed to the oracle, causing agents that attempt to decode and validate this data to revert when processing historical updates.

BVSS

AO:A/AC:L/AX:L/C:N/I:L/A:N/D:N/Y:N/R:N/S:U (2.5)

Recommendation

Add validation of `additionalData` format at storage time in `updateLatestRiskRoundData()`. For each `updateType`, decode and validate the `additionalData` against its expected schema before storing it. For example, if `updateType == "FREEZE_STATE"`, verify that `additionalData` can be successfully decoded as `(int256 lowerBound, int256 upperBound, uint256 freezeState)` and that the values satisfy logical constraints such as `upperBound > lowerBound` and `freezeState ∈ {0, 1}`.

Remediation Comment

SOLVED: The LlamaRisk team solved this finding.

Remediation Hash

<https://github.com/llama-risk/llamaguard-contracts/pull/19/>

7.4 STALE REPORT PROCESSING WITHOUT EXPIRATION VALIDATION

// LOW

Description

In `AbstractCreReceiver`, the `onReport` function validates metadata and report independently without ensuring temporal consistency between them. While the team confirms integration contracts prevent manipulation, the current design allows processing of arbitrarily old reports as if they were fresh updates. This is particularly problematic in `LlamaGuardOracle`, where `updateLatestRiskRoundData` accepts an `UpdateInput` without any deadline or round ID reference, enabling a stale report to overwrite current state downstream. A misconfigured system could replay older, valid reports to reset oracle parameters to outdated values, disrupting dependent protocols that rely on the assumption that processed updates reflect current market conditions.

BVSS

AO:A/AC:L/AX:L/C:N/!L/A:N/D:N/Y:N/R:N/S:U (2.5)

Recommendation

- **Add expiration validation:** Include a `deadline` field in the report payload that `onReport` verifies against `block.timestamp`, rejecting any report older than a configurable time window (e.g., 1 hour).
- **Implement round ID binding:** Modify `UpdateInput` to optionally include an expected `roundId` or `lastProcessedUpdateId`, allowing `updateLatestRiskRoundData` to validate that the update targets the correct state and detect replays of updates meant for earlier rounds.

Remediation Comment

SOLVED: The LlamaRisk team solved this finding.

Remediation Hash

<https://github.com/llama-risk/llamaguard-contracts/pull/19/changes/d95ab291e164990a7fd9eb25aa8bd7744fa9bccc>

7.5 ORACLE WRITE ACCESS NOT VALIDATED DURING INITIALIZATION

// INFORMATIONAL

Description

The `LlamaGuardOracleProxy` constructor accepts a `llamaGuardOracleAddress` parameter and assigns it directly to the `llamaguardOracle` state variable after only checking for a non-zero address. However, the `setLlamaGuardOracle()` function performs an additional critical validation by requiring that the oracle grants `WRITER_ROLE` to the proxy via `hasWriteAccess()` before accepting the new address. This inconsistency means the proxy could be initialized with an oracle instance that does not actually authorize the proxy to write data, resulting in a non-functional contract that accepts Chainlink CRE reports but fails silently when attempting to process them via `_processReport()`. An operator could deploy the proxy with an incompatible oracle and only discover the issue when the first workflow report arrives.

BVSS

AO:S/AC:L/AX:L/R:N/S:U/C:N/A:N/I:L/D:N/Y:N (0.5)

Recommendation

Add the same `hasWriteAccess()` validation in the constructor that is performed in `setLlamaGuardOracle()`. The constructor should require that `ILlamaGuardOracle(llamaGuardOracleAddress).hasWriteAccess(address(this))` returns true before assigning the oracle address, reverting with `InvalidLlamaGuardOracle()` if the validation fails. This ensures that the proxy is only deployed when properly configured and authorized to interact with the target oracle.

Remediation Comment

ACKNOWLEDGED: The LlamaRisk team acknowledged this finding.

7.6 MISSING CROSS-PARAMETER VALIDATION ON INDIVIDUAL SETTERS

// INFORMATIONAL

Description

The `ParameterRegistry` contract validates cross-parameter relationships only in `setParametersForAsset()`, which ensures all parameters are logically consistent when set together. However, the individual setter functions (`setMaxExpectedApy()`, `setUpperBoundTolerance()`, `setLowerBoundTolerance()`, `setMaxDiscount()`) only validate their single parameter against hardcoded limits and do not check relationships with other parameters in the `AssetConfig`. This allows operators to create nonsensical configurations where `lowerBoundTolerance` exceeds `upperBoundTolerance`, or where bounds are set to values that conflict with `maxDiscount` logic. Such configurations could cause off-chain oracle systems to malfunction or produce undefined behavior when consuming these parameters, and may result in incorrect risk assessments or frozen collateral positions.

BVSS

AO:S/AC:L/AX:L/R:N/S:U/C:N/A:N/I:L/D:N/Y:N (0.5)

Recommendation

Add cross-parameter validation to all individual setter functions that verifies parameter relationships remain consistent. For instance, `setLowerBoundTolerance()` should require `lowerBoundTolerance <= upperBoundTolerance`, and both tolerance setters should validate against `maxDiscount` if there is an intended relationship. Alternatively, implement a private validation helper function that checks all parameter relationships and reuse it across both `setParametersForAsset()` and each individual setter to ensure consistency is enforced uniformly. Document the intended relationships between tolerance, discount, and APY parameters so that future maintenance preserves these invariants.

Remediation Comment

ACKNOWLEDGED: The LlamaRisk team acknowledged this finding.

7.7 UPDATE TYPE REMOVAL FUNCTIONALITY MISSING

// INFORMATIONAL

Description

The `LlamaGuardOracle` contract provides no mechanism to remove invalid or deprecated update types. While `addUpdateType()` allows administrators to register new update types by adding them to both the `_validUpdateTypes` mapping and the `updateTypes` array, there is no corresponding removal function. This creates a permanent accumulation of update types in the `updateTypes` array and prevents deactivation of outdated or erroneous types. An administrator cannot remediate a mistakenly added update type, and the public `updateTypes` array becomes increasingly polluted with unused entries, potentially causing issues for integrators relying on this array for discovery.

BVSS

AO:A/AC:L/AX:L/C:N/I:N/A:N/D:N/Y:N/R:N/S:U (0.0)

Recommendation

Implement a `removeUpdateType()` function with `DEFAULT_ADMIN_ROLE` access that removes the update type from both `_validUpdateTypes` and the `updateTypes` array. The removal from the array should use the swap-and-pop pattern to maintain efficiency, or alternatively, iterate to find and remove the element if gas efficiency is not critical. Additionally, consider adding validation to prevent removal of update types that are still actively referenced in recent update history, or document the implications of removal for existing data integrity.

Remediation Comment

SOLVED: The LlamaRisk team solved this finding.

Remediation Hash

<https://github.com/llama-risk/llamaguard-contracts/pull/19>

7.8 REPORT VALIDATION CANNOT BE DISABLED

// INFORMATIONAL

Description

The `AbstractCreReceiver` contract declares `isReportWriteSecured` as a public constant with value `true`, making it immutable and unable to be modified after deployment. The `onReport()` function uses this constant in a conditional check that determines whether metadata validation is enforced. Since the compiler optimizes constant conditionals at compile time, the validation logic is permanently baked into the contract bytecode and cannot be toggled via any admin function. This design prevents operators from disabling security checks without redeploying the entire contract, and contradicts the apparent intent of having configurable workflows via `setWorkflowActive()`.

BVSS

AO:A/AC:L/AX:L/C:N/I:N/A:N/D:N/Y:N/R:N/S:U (0.0)

Recommendation

Convert `isReportWriteSecured` from a constant to a mutable state variable with a corresponding admin function to enable or disable report validation if required by business logic needs.

Remediation Comment

ACKNOWLEDGED: The LlamaRisk team acknowledged this finding.

Halborn strongly recommends conducting a follow-up assessment of the project either within six months or immediately following any material changes to the codebase, whichever comes first. This approach is crucial for maintaining the project's integrity and addressing potential vulnerabilities introduced by code modifications.